



caseSetup-v4.1.1 Technical User Guide

S. Chen

Jan 2026

This document covers the use of caseSetup for external automotive aerodynamics cases.

All materials within this module is confidential, do not copy, share or distribute in any manner without explicit permission from Ontario Tech Racing.

Contents

1	Introduction	4
2	Directory Structure	4
3	Basic Setup	6
3.1	Rotating Geometries	6
3.2	Controlling Run Settings	7
3.3	Controlling Simulation Settings	8
3.4	Boundary Conditions	9
3.5	Decomposition and Cluster Control	10
3.6	Fluid Properties	10
3.7	Meshing Control	10
3.8	Post Processing Control	11
3.9	Ride Height Control	12
3.9.1	Ride Height Map Generation	12
3.9.2	Ride Height Tool Operation	12
3.9.3	Ride Height Tool Usage	13
4	Prefix Controls	14
4.1	Rotating Geometries	14
4.2	Porous Media	15
5	Ride-Height and Suspension Kinematics	16
5.1	Ride-Height Setup	16
5.2	The Ride-Height File	17
5.3	Steering	17
5.4	Suspension Hardpoint File	17
5.5	Component PID Keywords	19
5.6	Optional Component Sections	19
5.7	Per-Component Transforms	20
5.8	Workflow Summary	20
5.9	Troubleshooting	21
6	Cornering (Single Reference Frame)	22
6.1	Cornering Setup	22
6.2	The Corner Centre	22
6.3	Wheel Rotation in a Corner	23
6.4	Cornering Driven by the Ride-Height Map	23
6.5	Domain Validity	23
6.6	Workflow Summary	23
6.7	Troubleshooting	24
7	Post-Processing Images (pvPost)	25
7.1	Inputs and Outputs	25
7.2	Running pvPost	25
7.3	The pvPostSetup File	25
7.3.1	[PV_POST_MAIN]	26
7.3.2	[SLICE]	26
7.3.3	[SURFACE] and [ISO_SURFACE]	27
7.4	Variables and Camera Views	27
7.5	Half-Symmetry Cases	27
7.6	Cornering (SRF) Cases	27
7.7	Performance	27

7.8	Workflow Summary	28
7.9	Troubleshooting	28

1 Introduction

`caseSetup` was built over a couple years for the Ontario Tech Racing team. This process is a mix of experiences gained from FSAE related activities and professional activities. The first iteration of `caseSetup` was born in the fall of 2022. The drive behind `caseSetup` was to improve the consistency of runs from user to user while reducing the chances of mistakes. The main idea behind it was to use pre-defined templates which mildly restricts the user from making un-intended modifications to the setup. This allows for consistent meshing, solving, and post processing regardless of user.

This document here is the technical user guide, while some Computational Fluid Dynamics (CFD) theories may be included, it is not meant to be an exhaustive explanation or discussion of the physics. Please refer to specialized CFD books and resources if more information is required.

2 Directory Structure

This process requires a specific directory structure for your cases to work. It looks like the following:

- `JOB-DIRECTORY`
 - `01_incoming(YYYY-MM-DD)`
 - * `2025-09-11_New_Geometry`
 - `02_references`
 - * `ANSA`
 - `trial009.ansa.gz`
 - * `MSH`
 - `trial009.obj.gz`
 - `fw-v1-1.obj.gz`
 - `03_reports`
 - * `051_064_063_report_2025-11-16.pptx`
 - `CASES`
 - * `001`
 - * `002`
 - * `003`
 - * `...`
 - * `009`
 - *
 - **`caseSetup`**
 - `constant`
 - `system`

The definitions for each folder is described here:

- `01_incoming(YYYY-MM-DD)` - Contains all the incoming geometry from other departments or other files which are required for the simulation. Each instance of CAD delivered to the department must be in a dated folder with a short descriptive name.
- `02_references` - Contains all the geometry files that are used in the simulation. Sub-directories are `ANSA` and `MSH`
- `ANSA` - Contains all the ANSA files that are used in the simulation, ideally contains the CAD files within them and not just the tri-surfaces. Should be saved as ".gz" for smaller file size.

- [MSH](#) - Contains all the tri-surfaces that are used in the simulation. Ideally this is where the original copies of all the STLs or OBJ files live. Each trial folder contains only a symbolic link referencing to this directory to reduce the number and size of geometry files.
- [03_reports](#) - Contains all the reports placed by the [pptGeneration](#) process or placed by the user manually.
- [CASES](#) - Contains all the simulation runs, this is where each case/trial folder lives, each trial is denoted by a 3 digit number XXX.
- [caseSetup](#) - This is the file that controls the simulations and the outputs.
- [system](#) - This directory contains the files required for the simulation to run.
- [constant](#) - Like [system](#) this directory also contains the files required for the simulation and also in [constant/triSurfaces](#) is where the geometry for the simulation is stored for this case. It is ideally a symbolic link so that repeated geometries are not copied many times in many cases.

Once all these directories are setup, we are ready to explore the rest of the features.

3 Basic Setup

To start, you would need a case directory created in your [CASES](#) directory. Once created, you will not have a [caseSetup](#) file just yet. To get a file, execute the following line in your case directory,

```
caseSetup --new
```

A fresh [caseSetup](#) file will be generated. The inside of the file will look like this:

```
[DEFAULTS]
DEFAULT_MODULES = defaultGlobalControl defaultGlobalSimControl defaultBCSetup defaultGlobalDecomposition
                  defaultGlobalMaterial defaultGlobalRefinement defaultPost
ADDON_MODULES =

[TITLES]
CASENAME = casename
CASEDESCRP = default description
JOBCODE = TS

[GEOMETRY]
GEOM = default.stl,1,8,5,1.1,high
```

The [DEFAULT_MODULES](#) line controls the settings that are available. If the setting is not visible, it just means that it will use the default setup. Each setting here is explained below:

- [CASENAME](#) - the case name for your case, it does not do anything at this time, but in the future it could be added to the post processing images.
- [CASEDESCRP](#) - case description, see [CASENAME](#)
- [JOBCODE](#) - Two letter initial for the job you are running, just helps identify the job that's running on the cluster queue.
- [GEOM](#) - List of geometries to use in the case. A comma separates each setting for that particular geometry, the next geometry goes on the subsequent line.
 - What each column represents:

```
GEOMETRY_FILE_NAME, SCALE, MESH_LEVEL, INFLATION_LAYERS, INFLATION_LAYER_GROWTH_RATE,
WALL_MODEL
```

Each geometry that is required in this run and is in its own file should be listed here, the settings shown above will be applied to all geometries in that file. If a different settings are required to be applied, that geometry should be broken out into it's own file. A common example is wheels.

3.1 Rotating Geometries

The wheels and tires are spinning and therefore should have a rotating wall boundary condition applied, therefore it **MUST** be in its own part. Example,

```
[DEFAULTS]
DEFAULT_MODULES = defaultGlobalControl defaultGlobalSimControl defaultBCSetup defaultGlobalDecomposition
                  defaultGlobalMaterial defaultGlobalRefinement defaultPost
ADDON_MODULES =

[TITLES]
CASENAME = casename
CASEDESCRP = default description
JOBCODE = TS

[GEOMETRY]
GEOM = body.obj.gz,1,8,5,1.1,high
      ROTA-fr-wh-lhs,1,8,5,1.1,high
      ROTA-fr-wh-rhs,1,8,5,1.1,high
      ROTA-rr-wh-lhs,1,8,5,1.1,high
      ROTA-rr-wh-rhs,1,8,5,1.1,high
```

Notice the prefix [ROTA](#), it lets [caseSetup](#) know that this geometry will have a rotating wall boundary condition applied. Upon running [caseSetup](#), if it's the first time with this set of geometry, [caseSetup](#) will stop and let you know that the following blocks have been added. The user should check that these are the appropriate settings for each wheel geometry.

NOTE: If using a plinth, ensure that the plinth is a separate geometry, else the automatic wheel center calculation will include the plinth in the radius calculation resulting in an incorrect radius and angular velocity!

```
[ROTA-fr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-fr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True
```

- **WH_RAD** `[[float,m]]` - wheel radius, distance from wheel center to ground (`r, default`)
- **WH_CENTER** `[[float,m]] [[float,m]] [[float,m]]` - wheel center coordinates in global coordinates system (`x y z, default`)
- **WH_AXIS** `[[float,m]] [[float,m]] [[float,m]]` - wheel axis of rotation from, using right hand rule with respect to the global coordinate system (`x y z, default`)
- **WH_AXIS** `[bool]` - activate rotating wall or use no-slip condition (`true, false`)

Using `default` option will trigger the automatic calculation but manual input of values is still possible. To speed up `caseSetup` it is recommended to copy the values output by the calculation in the terminal and input them into the `caseSetup` if coordinates and axis values are not expected to change for subsequent runs.

3.2 Controlling Run Settings

Deleting the module `defaultGlobalControl` will expose the following block:

```
[GLOBAL_CONTROL]
STARTFROM = latestTime
STARTTIME = 0
ENDTIME = 1000
DT = 1
WRITEINT = 500
PURGEINT = 1
AVGSTART = 500
```

- **STARTFROM** `[str]` - which time to start the simulation at, `latestTime` will start the simulation at the latest available time if running for an existing solution, `startTime` will have the simulation start at the value specified in **STARTTIME** (`latestTime, startTime`)
- **STARTTIME** `[float/int,s]` - time to start the simulation at, will be used if **STARTFROM** is specified as `startTime`, must use `int` if steady run is used, or `float` if a transient run is selected
- **ENDTIME** `[float/int,s]` - final time or time step to end the simulation at, must use `int` if steady run is used, or `float` if a transient run is selected
- **DT** `[float/int,s]` - timestep size, must use `1` if steady run is used, or any `float` if a transient run is selected

- **WRITEINT** [float/int,s] - interval which the data is written to disk for restart or post-processing use, must use **int** if steady run is used, or any **float** if a transient run is selected (**CAUTION:** saving too frequently can cause increased solve time and increased storage usage!)
- **PURGEINT** [int] - number of written timesteps to keep, will only keep **PURGEINT** number of time steps (aside from **STARTTIME** or 0)
- **DT** [float/int,s] - when to start averaging flow variables, must use **int** if steady run is used, or **float** if a transient run is selected

3.3 Controlling Simulation Settings

Deleting the module **defaultGlobalSimControl** will expose the following block:

```
[GLOBAL_SIM_CONTROL]
SIM_TYPE = steady
SIM_INIT = potential
SIM_SYM = half
TURB_MODEL = kosst
INIT_P = default
INIT_K = default
INIT_OMEGA = default
INIT_NUT = default
INIT_NUTILDA = default
```

- **SIM_TYPE** [str] - what type of simulation (**steady**, **transient**)
- **SIM_INIT** [str] - what type of initialization, **steady** only available when **SIM_TYPE** = **transient** (**none**, **potential**, **steady**)
- **SIM_SYM** [str] - simulation symmetry type, recommended to run in **half** if not doing yaw and vehicle is mostly symmetrical, use **full** if running internal flow (**half**, **full**)
- **TURB_MODEL** [str] - turbulence model applied, **transient** runs can only use **sa** ([**kosst**,**sa**])
 - **steady**
 - * **kosst** - $k - \omega$ Shear Stress Transport ($k - \omega$ SST)
 - * **sa** - Spalart-Allmaras
 - **transient**
 - * **sa** - Spalart-Allmaras Delayed Detached Eddy Simulation (SADDES)
- **INIT_P** [float,Pa] - initial conditions for pressure, recommended to leave in default unless specific values are required (**float**, **default**)
- **INIT_K** [float,m2s-2] - initial conditions for turbulent kinetic energy, recommended to leave in default unless specific values are required (**float**, **default**)
- **INIT_OMEGA** [float,s-1] - initial conditions for ω term, recommended to leave in default unless specific values are required (**float**, **default**)
- **INIT_NUT** [float,m2s-1] - initial conditions for ν_t term, recommended to leave in default unless specific values are required (**float**, **default**)
- **INIT_NUTILDA** [float,m2s-1] - initial conditions for $\tilde{\nu}$ term, recommended to leave in default unless specific values are required (**float**, **default**)

3.4 Boundary Conditions

Deleting the module `defaultBCSetup` will expose the following block:

```
[BC_SETUP]
INLET_MAG = 15
YAW = 0
DRAG_VEC = default
LIFT_VEC = default
PITCH_VEC = default
GROUND = moving
REFCOR = 0.775 0 0
REFLEN = 1.55
REFWIDTH = 1
REFAREA = 1
POROSITY = yes
BASE_CELL_SIZE = 1.6
DOMAIN_SIZE = 48 16 16
DOMAIN_OFFSET = 0.5
FRT_WH_CTR = 0 0 0
```

- `INLET_MAG` [`float`,`ms-1`] - inlet velocity magnitude ([`float`])
- `YAW` [`float`,`deg`] - yaw angle, will only yaw in the positive direction, if `SIM_SYM` is `half`, this value not used (`float`)
- `DRAG_VEC` [[`float`,`m`] [`float`,`m`] [`float`,`m`]] - drag force vector, following SAE J1594 convention by default (`x y z`, `default`)
- `LIFT_VEC` [[`float`,`m`] [`float`,`m`] [`float`,`m`]] - lift force vector, following SAE J1594 convention by default (`x y z`, `default`)
- `PITCH_VEC` [[`float`,`m`] [`float`,`m`] [`float`,`m`]] - pitch moment vector, following SAE J1594 convention by default (`x y z`, `default`)
- `GROUND` [`str`] - ground plane boundary condition (`moving`,`stationary`)
- `REFCOR` [[`float`,`m`] [`float`,`m`] [`float`,`m`]] - point defining center of rotation, typically center of wheel projected to the ground (`x y z`)
- `REFLEN` [[`float`,`m`]] - reference length, typically wheelbase length (`float`)
- `REFWIDTH` [[`float`,`m`]] - reference width, typically trackwidth length (`float`)
- `REFAREA` [[`float`,`m`]] - reference area, typically frontal area or `1` to have calculations done in $C_D A$ (`float`)
- `POROSITY` [`str`] - to calculate forces including forces due to porous media, recommend to leave at `yes` (`yes`, `no`)
- `BASE_CELL_SIZE` [[`float`,`m`]] - base cell size of blockMesh, resulting mesh levels are calculated based off this number, recommend to leave at `1.6` (`float`)
- `DOMAIN_SIZE` [[`float`,`m`] [`float`,`m`] [`float`,`m`]] - dimensions for blockMesh domain (`x y z`)
- `DOMAIN_OFFSET` [[`float`,`m`]] - distance of `FRT_WH_CTR` to the inlet boundary as a ratio of domain `x` length, i.e. `0.5` means the `FRT_WH_CTR` is located at the center of the domain (`float`)
- `FRT_WH_CTR` [[`float`,`m`] [`float`,`m`] [`float`,`m`]] - coordinate for the center point projected to the ground of the front wheel axle (`x y z`)

3.5 Decomposition and Cluster Control

Deleting the module `defaultGlobalDecomposition` will expose the following block:

```
[GLOBAL_COMPUTE_SETUP]
QUEUE_TYPE = slurm
SCRIPT_VERSION = 1
NUM_NODES = 1
TASK_PER_NODE = 72
DECOMP = 18 2 2
```

- `QUEUE_TYPE` [`str`] - NOT ACTIVE, FOR FUTURE USE (`str`)
- `SCRIPT_VERSION` [`int`] - NOT ACTIVE, FOR FUTURE USE (`int`)
- `NUM_NODES` [`int`] - NOT ACTIVE, FOR FUTURE USE (`int`)
- `TASK_PER_NODE` [`int`] - number of cores to use (`int`)
- `DECOMP` [`int int int`] - decomposition pattern, dictates the number of divisions in each direction during decomposition, freestream direction should be the largest, the product of the 3 values must match `TASK_PER_NODE` (`x y z`)

3.6 Fluid Properties

Deleting the module `defaultMaterial` will expose the following block:

```
[GLOBAL_MATERIAL]
NPHASE = 1
MATERIAL_TYPES = air
VISCOSITY = 1.46e-5
```

- `NPHASE` [`int`] - NOT ACTIVE, FOR FUTURE USE (`int`)
- `MATERIAL_TYPES` [`str`] - NOT ACTIVE, FOR FUTURE USE (`str`)
- `VISCOSITY` [`float`, `m2s-1`] - fluid kinematic viscosity (`float`)

3.7 Meshing Control

Deleting the module `defaultGlobalRefinement` will expose the following block:

```
[GLOBAL_REFINEMENT]
TEMPLATE_TYPE = snappyTemplate1
MIN_GEOM_LEVEL = 8
REF_FEAT_ANGLE = 25
GEOM_REF_DIST = 0.01 0.05 0.1
GEOM_REF_LEVELS = 9 8 7
DEFAULT_WAKE_REF = defaultWake
LOC_IN_MESH = -1 -1 4
LAYER_TYPE = absolute
DEF_EX_RATIO = 1.2
REF_ANGLE = 25
FEAT_EDGE_ANGLE = 140
FEAT_EDGE_LEVEL_INC = 1
FIRST_LAYER_HEIGHT = 0.0001
FINAL_LAYER_HEIGHT = 0.001
FINAL_LAYER_RATIO = 0.3
```

- `TEMPLATE_TYPE` [`str`] - meshing template to use (`str`)
 - `snappyHexMesh` - template file must exist in `setupTemplates/<caseSetup_template>/defaultDicts/`
 - `ansaMesh` - `TEMPLATE_TYPE` string must include `caseSetup` template name as well as `ansa` separated by underscores i.e. `otr_ansa_default`. The `ansa` is used to trigger the use of `ansaMesh`. The template must be located in `setupTemplates/<caseSetup_template>/defaultANSA/`. The last value entry in `TEMPLATE_TYPE` is the name of the template within that path and can be of any length. Just NO SPACES!

- **MIN_GEOM_LEVEL** [**int**] - coarses mesh level to be used for all geometries in the case (**int**)
- **REF_FEAT_ANGLE** [**float,deg**] - angle where curvatures with angles larger than it would trigger mesh refinement in that area, operates only for **ansaMesh** (**float**)
- **GEOM_REF_DIST** [[**float,m**] [**float,m**] [**float,m**] ...] - distance from the surface which the mesh level from **GEOM_REF_LEVELS** shall be applied, unlimited entries, but must match length in **GEOM_REF_LEVELS** (**float float ...**)
- **GEOM_REF_LEVELS** [**int int ...**] - refinement levels for each corresponding distance in **GEOM_REF_DIST** unlimited entries, but must match length in **GEOM_REF_DIST** (**int int ...**)
- **DEFAULT_WAKE_REF** [**str**] - wake refinement generated by default or use specific template (**true, false, str**)
 - if using specific template, it should exist in `setupTemplates/<caseSetup_template>/defaultRefinements/`
- **LOC_IN_MESH** [[**float,m**] [**float,m**] [**float,m**]] - point which defines the "inside" location of the domain, only works for **snappyHexMesh** (**float float**)
- **LAYER_TYPE** [**str**] - how the layer growth occurs globally, only works for **ansaMesh**, defined in **snappyHexMesh** by template (**absolute, aspect**)
 - **absolute** - first layer height is defined absolutely based on **FIRST_LAYER_HEIGHT**, rest of inflation layer sizing determined by aspect ratio set for the geometry
 - **aspect** - first layer height is defined as a ratio to cell size outside of the inflation layers, global settings currently hardcoded as **0.5**, rest of inflation layer sizing determined by aspect ratio set for the geometry
- **DEF_EX_RATIO** [**float**] - global layer expansion ratio (**float**)
- **REF_ANGLE** [**float,deg**] - angle where curvatures with angles larger than it would trigger mesh refinement in that area, operates only for **snappyHexMesh** (**float**)
- **FEAT_EDGE_ANGLE** [**float,deg**] - angle under which triggers edge refinement, only works for **snappyHexMesh** (**float**)
- **FEAT_EDGE_LEVEL_INC** [**int**] - refinement level increment for edges defined by **FEAT_EDGE_ANGLE**, only works for **snappyHexMesh** (**int**)
- **FIRST_LAYER_HEIGHT** [[**float,m**]] - first layer height, only works for **ansaMesh** (**float**)
- **FINAL_LAYER_HEIGHT** [[**float,m**]] - total height of inflation layers, not currently operational, for future use (**float**)
- **FINAL_LAYER_RATIO** [[**float,m**]] - ratio of last layer and the surrounding element size, not currently operational, for future use (**float**)

3.8 Post Processing Control

Deleting the module **defaultPost** will expose the following block:

```
[GLOBAL_POST_PRO]
ISOQ = 2000
ISOCPT = 0
SLICES = True
XSLICE = default
YSLICE = default
ZSLICE = default
```

- **ISOQ** [**float,s-2**] - Q criterion value of which to generate VTK iso surfaces of (**float**)

- **ISOCP** [float] - $C_{p,total}$ value of which to generate VTK iso surfaces of (float)
- **SLICES** [str] - to generate VTK slices (true, false)
- **XSLICE** [[float,m] [float,m] [float,m]] - start, end and spacing of the x normal slices (float, float, float)
- **YSLICE** [[float,m] [float,m] [float,m]] - start, end and spacing of the y normal slices (float, float, float)
- **ZSLICE** [[float,m] [float,m] [float,m]] - start, end and spacing of the z normal slices (float, float, float)

3.9 Ride Height Control

Deleting the module `defaultRideHeight` will expose the following block that is under development currently. The goal is to create a process which can do ride height mapping.

```
[RIDE_HEIGHT_SETUP]
RUN_RIDE_HEIGHT = false
RIDE_HEIGHT_FILE =
USE_DEPEND = True
```

This section outlines the usage of the ride height mapping tool. A ride height map is useful to understand the dynamic sensitivity of an aero package as the car goes through the corners. We can use it to run in virtual lap time simulations as well. The use of the ride height mapping tool is diverse, we can use it for evaluation of a concept to understand the overall sensitivity from concept to concept. Some concepts perform better under braking, or under acceleration.

For the sake of simplicity, we make the assumption that there is enough time for the air flow to settle and there is no hysteresis from attitude to attitude on the track, although this might not be true. Therefore, each ride height is run statically with no movement in the simulation after transformation.

3.9.1 Ride Height Map Generation

Generating a ride height map is done by using on track or simulations data, looking at the front ride height vs rear ride height scatter plots and creating a point grid that encompasses the data. There might be some outliers that rarely get reached, but it is helpful to add a point there to ensure the map is interpolated well. A full ride height map can be many points, which can result in very long total computational times. They should not be something run day to day. Rather a limit selection of ride heights can be run for conceptual analysis. The full map can be run at the end of a design cycle to evaluate the overall sensitivity.

For day to day usage, the ride height points can be limited to 3-5 points relevant to the geometry concept at hand. For example, a front wing change it might be more relevant to use select nose down and nose down plus roll conditions in addition to a level condition.

3.9.2 Ride Height Tool Operation

The ride height tool operates by reading a `.csv` file containing front left (`fl`), front right (`fr`), rear left (`rl`), and rear right (`rr`) ride heights as a deviation from the baseline condition. The ride height translation is done with a car centric coordinate system, meaning the car body does not move, only the wheels do as well as the domain. This gives the easiest comparison between ride heights in post processing. The ride height reference points are currently referenced to the axle location in x and z directions at the wheel base center in y . This is also assuming your front and rear wheelbases are the same. Future iterations will include alternate reference point locations.

Additionally, the model used for the ride height mapping should not contain suspension components as they will not be transformed at this time. Future iterations might include suspension kinematic transformations.

3.9.3 Ride Height Tool Usage

To use the ride height tool, delete the `defaultRideHeight` module in the `DEFAULT_MODULES` line in `caseSetup`. This will reveal the ride height module in `caseSetup`.

4 Prefix Controls

There are 3 or 4 letter prefixes that allow special control of your geometry and simulation. These help with assigning porous media regions, Multiple Reference Frame (MRF) regions, or even more control over meshing for certain geometries.

4.1 Rotating Geometries

The **ROTA** prefix allows the assignment of rotating wall to geometries such as wheels. A usage example is shown below.

```
[DEFAULTS]
DEFAULT_MODULES = defaultGlobalControl defaultGlobalSimControl defaultBCSetup defaultGlobalDecomposition
                  defaultGlobalMaterial defaultGlobalRefinement defaultPost
ADDON_MODULES =

[TITLES]
CASENAME = casename
CASEDESCRP = default description
JOBCODE = TS

[GEOMETRY]
GEOM = body.obj.gz,1,8,5,1.1,high
      ROTA-fr-wh-lhs,1,8,5,1.1,high
      ROTA-fr-wh-rhs,1,8,5,1.1,high
      ROTA-rr-wh-lhs,1,8,5,1.1,high
      ROTA-rr-wh-rhs,1,8,5,1.1,high
```

Notice the prefix **ROTA**, it lets **caseSetup** know that this geometry will have a rotating wall boundary condition applied. Upon running **caseSetup**, if it's the first time with this set of geometry, **caseSetup** will stop and let you know that the following blocks have been added. The user should check that these are the appropriate settings for each wheel geometry.

NOTE: If using a plinth, ensure that the plinth is a separate geometry, else the automatic wheel center calculation will include the plinth in the radius calculation resulting in an incorrect radius and angular velocity!

```
[ROTA-fr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-fr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-lhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True

[ROTA-rr-wh-rhs]
WH_RAD = default
WH_CENTER = default
WH_AXIS = default
ROT_WH = True
```

- **WH_RAD** `[[float,m]]` - wheel radius, distance from wheel center to ground (**r**, **default**)
- **WH_CENTER** `[[float,m]] [[float,m]] [[float,m]]` - wheel center coordinates in global coordinates system (**x y z**, **default**)
- **WH_AXIS** `[[float,m]] [[float,m]] [[float,m]]` - wheel axis of rotation from, using right hand rule with respect to the global coordinate system (**x y z**, **default**)
- **WH_AXIS** `[bool]` - activate rotating wall or use no-slip condition (**true**, **false**)

Using **default** option will trigger the automatic calculation but manual input of values is still possible. To speed up **caseSetup** it is recommended to copy the values output by the calculation in the terminal and input them into the **caseSetup** if coordinates and axis values are not expected to change for subsequent runs.

4.2 Porous Media

When radiators are used in a simulation, it would be very computationally expensive to model each radiator fin. So instead we use the Darcy-Forcheimer porous media calculation to model the pressure drop across a radiator which allows us to estimate the mass flow across the radiator and resulting forces.

The prefix **POR** is used to define the porous media geometry. **CAUTION:** The porous media geometry must only contain the "box" or region that represents the radiator fin region, do not include the frame or any other part of the radiator. The box which represents the fin region should be a simple prism. The inlet and outlets should have their own PID with the keyword **-inlet** or **-outlet**. There should only be 1 of each PID!

The mass flow calculations are performed on each pid in the porous media, therefore including the walls or any other surfaces where flow isn't moving *across* would make the calculations erroneous.

Including the prefix **POR** on a geometry would expose the following block:

```
GEOM = trial005-rad.obj.gz,1,8,6,1.4,high
      fw-H05A00.obj.gz,1,8,6,1.4,high
      rw-R11I01.obj.gz,1,8,6,1.4,high
      POR-rad-001.obj.gz,1,10,6,1.4,high
      ROTA-fr-wh-lhs-v3-3.obj.gz,1,8,6,1.4,high
      ROTA-fr-wh-rhs-v3-3.obj.gz,1,8,6,1.4,high
      ROTA-rr-wh-lhs-v3-3.obj.gz,1,8,6,1.4,high
      ROTA-rr-wh-rhs-v3-3.obj.gz,1,8,6,1.4,high

[POR-rad-001]
DCOEFFS = 1000
FCOEFFS = 1000
VEC1 = 0.7071 0 -0.7071
VEC2 = 0 1 0
POINT = 1.7875 -0.000583 0.342739
SAMPLE_POR = True
```

the name of the block will be the name of the geometry just without the extension to identify it.

- **DCOEFFS** [float] - Darcy coefficient attained from experimental testing of pressure drop vs flow rate of the radiator (float)
- **FCOEFFS** [float] - Forcheimer coefficient attained from experimental testing of pressure drop vs flow rate of the radiator (float)
- **VEC1** [[float,m] [float,m] [float,m]] - vector defining the normal of the radiator exit (float float float)
- **VEC2** [[float,m] [float,m] [float,m]] - vector in plane with the radiator exit (float float float)
- **POINT** [[float,m] [float,m] [float,m]] - point defining the "inside" of the radiator (float float float)
- **SAMPLE_POR** [str,m] - to sample the mass flow through the radiator (true, false)

5 Ride-Height and Suspension Kinematics

`caseSetup` can automatically generate a family of child cases, each at a different ride-height state, and deform the geometry to follow the actual suspension motion. There are two ways this is done:

- `simple` - the wheels (and any `ROTA` geometry) are translated vertically by the requested corner displacement. No suspension linkage is required.
- `kinematic` - a double wishbone and pushrod kinematic solver computes the realistic motion of every suspension component (wheel camber and toe, control arm rotation, rocker angle, damper stroke) and applies a per-component transform to the matching parts of the geometry.

The mode is selected by the `USE_KINEMATIC_SOLVER` flag described below.

5.1 Ride-Height Setup

Deleting the module `defaultRideHeight` will expose the following block:

```
[RIDE_HEIGHT_SETUP]
RUN_RIDE_HEIGHT = False
RIDE_HEIGHT_FILE =
RUN_RH_POINTS = 1 2 3 4 5
USE_DEPEND = True
RH_UNIT = mm
INIT_RH =
RH_REF_WIDTH =
RH_REF_LEN =
USE_KINEMATIC_SOLVER = False
HARDPOINT_FILE = rideHeightUtils/suspensionHardpoints_template.cfg
HARDPOINT_SCALE = 0.001
SUSP_REQUIRE_ALL_PIDS_MATCHED = False
```

- `RUN_RIDE_HEIGHT` [bool] - master switch, when `True`, running `caseSetup` will generate one child case per requested ride-height point (`True`, `False`)
- `RIDE_HEIGHT_FILE` [str] - name of the ride-height file describing each point, see Section [The Ride-Height File](#) (str)
- `RUN_RH_POINTS` [int] - space separated list of point indices, matching the `point` column of the ride-height file, that will actually be built, allows running a subset of the table (`int int ...`)
- `USE_DEPEND` [bool] - reuse child case dependencies and links when they are already present (`True`, `False`)
- `RH_UNIT` [str] - unit of the corner displacements in the ride-height file (`mm`, `m`)
- `INIT_RH` [float] - optional initial ride-height offset applied to all points (`float`)
- `RH_REF_WIDTH` [[float,m]] - optional reference track width used when converting corner displacements into pitch, roll and heave (`float`)
- `RH_REF_LEN` [[float,m]] - optional reference wheelbase used when converting corner displacements into pitch, roll and heave (`float`)
- `USE_KINEMATIC_SOLVER` [bool] - enable the double wishbone and pushrod solver and the per-component geometry deformation, when `False` only simple vertical wheel translation is applied (`True`, `False`)
- `HARDPOINT_FILE` [str] - path to the suspension hardpoint definition file, required when `USE_KINEMATIC_SOLVER` is `True` (str)
- `HARDPOINT_SCALE` [float] - multiplier applied to every hardpoint coordinate to convert it to metres, use `0.001` if the hardpoint file is written in millimetres or `1.0` if it is already in metres (`float`)

- `SUSP_REQUIRE_ALL_PIDS_MATCHED` [bool] - when `True`, `caseSetup` stops with an error if any geometry part fails to match a suspension component, when `False` unmatched parts are left untouched and a warning is logged (`True`, `False`)

To run a ride-height study, set `RUN_RIDE_HEIGHT` to `True` and run `caseSetup` as normal. Child cases are then created and built automatically.

NOTE: `-rideHeightMode` is an internal flag used by `caseSetup` when it re-invokes itself inside each generated child case, it should not be supplied by the user.

5.2 The Ride-Height File

The ride-height file is a comma separated table where each row is one suspension state. The corner columns are the vertical wheel displacements in the unit set by `RH_UNIT`.

```
point,fl,fr,rl,rr,yaw,steer
1,0,0,0,0,0,0
2,-10,-10,0,0,0,5
3,-20,-20,0,0,0,0
4,0,-40,10,-30,0,0
5,-10,-10,-10,-10,0,0
```

- `point` [int] - unique index for the state, referenced by `RUN_RH_POINTS` and used to name the child case, for example `001\_2` (int)
- `fl, fr, rl, rr` [[float]] - front-left, front-right, rear-left and rear-right wheel vertical displacements, negative values typically indicate compression (bump) and positive values rebound (droop) (float)
- `yaw` [[float,deg]] - vehicle yaw angle for that state, `0` for straight-line (float)
- `steer` [[float,deg]] - optional front road-wheel steering angle for that state, see Section `Steering`, `0` or omitted for no steer (float)

5.3 Steering

When the kinematic solver is enabled, an optional `steer` column in the ride-height file drives the front suspension through its steering rack. The value is the road-wheel angle, in degrees, requested at the reference wheel (the front-left). `caseSetup` solves for the rack travel that produces that angle at the front-left, then applies the same rack travel to the front-right. Because both front wheels share one rack but sit at different ride heights, the front-right angle is solved from the geometry rather than copied, so the natural Ackermann difference between the two wheels appears automatically. The rear wheels are not steered.

The steer is a true rack motion, not a rigid wheel rotation. The tie rod inner (rack) joint slides along the rack axis, the tie rod outer joint follows the upright, and the tie rod length is preserved. The wheel, upright and tie rod therefore move as a real linkage, and the resulting wheel pose feeds the same per-component transforms used for ride height.

The rack pushes the tie rod inner joint along the lateral (`y`) direction by default. This can be overridden per corner with the optional `RACK_AXIS` key in the hardpoint file. If the `steer` column is absent or all zero, the solver behaves exactly as a passive ride-height study.

NOTE: The `steer` value is the absolute road-wheel angle at the front-left, not an offset added to the static toe. A positive value steers the front-left wheel so its forward direction rotates toward `+y`, flip the sign in the ride-height file to steer the other way.

5.4 Suspension Hardpoint File

When the kinematic solver is enabled, the geometry of each corner's linkage is described in the hardpoint file pointed to by `HARDPOINT_FILE`. A template is provided at `rideHeightUtils/suspensionHardpointFileTemplate.txt`. It uses the same format as the `caseSetup` file, with one section per corner, `[SUSP_FL]`, `[SUSP_FR]`, `[SUSP_RL]` and `[SUSP_RR]`. Each corner section looks like this:

```

[SUSP_FL]
WHEEL_CENTER_STATIC = 0 -781.03 356.57
UCA_F_INNER = -64.67 -157.20 500.45
UCA_R_INNER = 417.86 -157.20 488.42
LCA_F_INNER = -89.37 -157.20 330.51
LCA_R_INNER = 416.49 -157.20 350.42
TIE_INNER = -127.85 -157.20 330.51
UCA_OUTER_STATIC = 33.03 -707.27 520.88
LCA_OUTER_STATIC = 12.36 -737.95 331.40
TIE_OUTER_STATIC = -58.25 -706.55 461.63
PUSHROD_OUTER_STATIC = 12.36 -737.95 331.40
ROCKER_PIVOT = 61.44 -77.24 544.58
ROCKER_AXIS = 1.0 0.0 0.0
ROCKER_PUSHROD_JOINT_REF = 61.09 -159.93 558.55
ROCKER_DAMPER_JOINT_REF = 61.44 -77.24 614.58
ROCKER_DAMPER_CHASSIS = 61.44 0 544.58
WHEEL_AXIS_LOCAL = 0.0 1.0 0.0
WHEEL_FORWARD_LOCAL = 1.0 0.0 0.0
FL_UCA_PID = fluca
FL_LCA_PID = fllca
FL_ROCKER_PID = flrocker
FL_PUSHROD_PID = flprod
FL_DAMPER_PID = fldamper
FL_TIE_PID = fltie
FL_WHEEL_PID = flwheel

```

All coordinates are given in the global coordinate system, in the units implied by [HARDPOINT_SCALE](#), for example millimetres when [HARDPOINT_SCALE](#) is 0.001.

- [WHEEL_CENTER_STATIC](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - static wheel center position ([x y z](#))
- [UCA_F_INNER](#), [UCA_R_INNER](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - upper control arm front and rear chassis (inner) mounts ([x y z](#))
- [UCA_OUTER_STATIC](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - upper control arm outer (upright) ball joint ([x y z](#))
- [LCA_F_INNER](#), [LCA_R_INNER](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - lower control arm front and rear chassis (inner) mounts ([x y z](#))
- [LCA_OUTER_STATIC](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - lower control arm outer (upright) ball joint ([x y z](#))
- [TIE_INNER](#), [TIE_OUTER_STATIC](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - tie rod inner (chassis or rack) and outer (upright) mounts ([x y z](#))
- [PUSHROD_OUTER_STATIC](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - pushrod to upright (or pushrod to lower control arm) joint ([x y z](#))
- [ROCKER_PIVOT](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - rocker rotation center ([x y z](#))
- [ROCKER_AXIS](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - rocker rotation axis, using the right hand rule ([x y z](#))
- [ROCKER_PUSHROD_JOINT_REF](#), [ROCKER_DAMPER_JOINT_REF](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - the points on the rocker where the pushrod and damper attach ([x y z](#))
- [ROCKER_DAMPER_CHASSIS](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - the chassis side damper mount ([x y z](#))
- [WHEEL_AXIS_LOCAL](#), [WHEEL_FORWARD_LOCAL](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - local spin axis and forward direction of the wheel, used to resolve camber and toe ([x y z](#))
- [RACK_AXIS](#) [\[\[float,m\] \[float,m\] \[float,m\]\]](#) - optional steering rack travel direction, the tie rod inner joint slides along this axis when a [steer](#) angle is requested, defaults to lateral 0 1 0 if omitted ([x y z](#))

5.5 Component PID Keywords

Any key in a corner section whose name contains **PID** is treated as a part name keyword. Its value is matched against the group or solid names inside the OBJ or STL geometry to decide which faces belong to which suspension component. The match is case insensitive and is satisfied when the keyword appears as a substring of the part name.

- **FL_WHEEL_PID** [str] - part keyword for the wheel (str)
- **FL_UCA_PID** [str] - part keyword for the upper control arm (str)
- **FL_LCA_PID** [str] - part keyword for the lower control arm (str)
- **FL_ROCKER_PID** [str] - part keyword for the rocker (str)
- **FL_PUSHROD_PID** [str] - part keyword for the pushrod (str)
- **FL_DAMPER_PID** [str] - part keyword for the damper (str)
- **FL_TIE_PID** [str] - part keyword for the tie rod (str)

The same set of keys exist for the other corners using the **FR**, **RL** and **RR** prefixes. The keyword is automatically classified into a component type. Classification accepts the full component name as well as common abbreviations, so values such as **flprod** or **rlpr** are correctly recognised as a pushrod. The recognised keywords for each component are:

- **UCA** - **uca**
- **LCA** - **lca**
- **ROCKER** - **rocker, rock, rkr**
- **PUSHROD** - **pushrod, prod, pshrd, push, pr**
- **DAMPER** - **damper, damp, shock, strut**
- **WHEEL** - **wheel, whl**
- **TIE** - **tie, tierod, trod**

NOTE: The keyword must be a substring of the actual part name in your geometry. For example a part group named **body_susp_fl1lca** is matched by the keyword **fl1lca**. If a part is not matched it is left in its static position. Set **SUSP_REQUIRE_ALL_PIDS_MATCHED** to **True** to turn unmatched parts into a hard error instead.

5.6 Optional Component Sections

Additional sections can attach extra geometry to a corner without that geometry being one of the primary kinematic parts. Any section that defines a **CORNER** key contributes its **PID** keywords to that corner. A usage example is shown below.

```
[SUSP_COMP_EXAMPLE_1]
CORNER = fl
PID = fluca_bracket
```

- **CORNER** [str] - which corner this geometry belongs to (**fl**, **fr**, **rl**, **rr**)
- **PID** [str] - part keyword for the extra geometry, any key whose name contains **PID** is accepted (str)

These sections do not introduce a transform of their own. They reuse the same machinery as the primary parts, so the choice of which component an optional part moves with is made in two steps:

- **Which corner** - given by the [CORNER](#) key. The section's [PID](#) keywords are merged into that corner's keyword pool, exactly as if they had been written directly inside the matching corner section, for example [\[SUSP_FL\]](#).
- **Which component** - given by the text of the keyword itself. The keyword is classified using the same recognised keywords listed under [Component PID Keywords](#), and the optional part inherits that component's translation, rotation and pivot.

In the example above, [fluca_bracket](#) contains [uca](#), so it is classified as an upper control arm and rides with the front-left upper control arm transform. In the same way a keyword such as [frdamper_shroud](#) contains [damper](#) and follows the front-right damper, including its length morph, and a keyword containing [wheel](#) follows the wheel center motion.

NOTE: If an optional keyword contains none of the recognised component keywords, for example [fl_aero_fairing](#), it is not classified into any component, so no transform is assigned and the part is copied unchanged. To make an optional part follow a component, ensure its keyword contains the relevant keyword such as [uca](#), [damper](#) or [wheel](#).

5.7 Per-Component Transforms

For each ride-height point and corner the solver produces a transform for every component, made up of a translation, a rotation about an axis and a pivot point. How each component moves is summarised below.

- [WHEEL](#) - translates by the wheel center displacement and rotates with the camber and toe from the solver, about the static wheel center
- [UCA](#) - rotates as its outer joint follows the wheel, about the midpoint of its inner chassis mounts
- [LCA](#) - rotates as its outer joint follows the wheel, about the midpoint of its inner chassis mounts
- [ROCKER](#) - rotates in place by the rocker angle about the rocker axis, about the rocker pivot
- [PUSHROD](#) - inherits the motion of the rocker, about the rocker pivot
- [DAMPER](#) - inherits the motion of the rocker and stretches by the damper stroke, about the rocker pivot
- [TIE](#) - rotates as its outer end follows the wheel, about the tie rod inner mount, and when steering the inner end slides along the rack axis

For every transformed geometry a log is written next to the output file, [*.susp_pid_transform.json](#) and [*.susp_pid_transform.csv](#), listing each part, the keyword it matched, and the applied translation and rotation. Inspect these to confirm the expected parts were moved.

5.8 Workflow Summary

- Set [RUN_RIDE_HEIGHT](#) to [True](#) and point [RIDE_HEIGHT_FILE](#) at your ride-height file.
- If kinematic motion is required, set [USE_KINEMATIC_SOLVER](#) to [True](#), copy and edit the hardpoint template, and set [HARDPOINT_FILE](#) and [HARDPOINT_SCALE](#).
- Ensure the geometry part names contain the PID keywords from the hardpoint file.
- Run [caseSetup](#) as normal. Child cases named [<case>_<point>](#) are created, the geometry is deformed, and [caseSetup](#) is re-run automatically inside each child.
- Each child case is then a standard, ready to mesh case at that suspension state.

5.9 Troubleshooting

- **PID scan skipped** - a geometry could not be processed by the PID aware path, for example an unsupported or binary format, so the file falls back to simple wheel translation. Check that the suspension geometry is ASCII OBJ or ASCII STL.
- **Unmatched PID** - the keyword from the hardpoint file does not appear in any part name. Confirm the keyword is a substring of the actual part name, or add a recognised keyword.
- **Geometry not moving as expected** - verify **HARDPOINT_SCALE** matches the unit of your hardpoint coordinates, and review the per-part transform log.
- **Kinematic solver skipped** - confirm **USE_KINEMATIC_SOLVER** is **True**, that **HARDPOINT_FILE** exists relative to the case, and that all required hardpoint keys are present.

6 Cornering (Single Reference Frame)

`caseSetup` can set up a steady curved-path (cornering) case using a Single Reference Frame (SRF). Instead of meshing a curved tunnel, the existing rectangular domain is solved in a frame that rotates at a constant angular velocity about a vertical axis through the corner centre. In that rotating frame the car follows a circular path of radius `CORNER_RADIUS` at the freestream speed `INLET_MAG`, and the steady flow field represents a mid-corner condition. The solver is `SRFSimpleFoam`.

6.1 Cornering Setup

Deleting the module `defaultCornering` will expose the following block:

```
[CORNERING_SETUP]
RUN_CORNERING = False
CORNER_RADIUS =
CORNER_DIR = left
CORNER_CENTER = default
CORNER_CLEARANCE_FACTOR = 3
CORNER_SOLVER = SRFSimpleFoam
```

- `RUN_CORNERING` [bool] - master switch, when `True` the case is built as an SRF cornering case (`True`, `False`)
- `CORNER_RADIUS` [[float,m]] - radius of the corner the car is negotiating, measured from the corner centre to the reference centre of rotation, the frame angular velocity is `INLET_MAG / CORNER_RADIUS` (float)
- `CORNER_DIR` [str] - direction of the turn, `left` places the corner centre on the car's left (-y) and `right` on the car's right (+y) (`left`, `right`)
- `CORNER_CENTER` [str or [float,m] [float,m] [float,m]] - corner centre location, `default` derives it from the reference centre of rotation with a lateral offset of `CORNER_RADIUS`, or supply three floats to set it explicitly (`default`, or `x y z`)
- `CORNER_CLEARANCE_FACTOR` [float] - minimum ratio of `CORNER_RADIUS` to the domain lateral half-width, guards against a turn so tight that the rectangular box is a poor model of the curved tunnel (float)
- `CORNER_SOLVER` [str] - the OpenFOAM solver used for the cornering run (`SRFSimpleFoam`)

To run a cornering case, set `RUN_CORNERING` to `True`, provide a valid `CORNER_RADIUS` and `CORNER_DIR`, and run `caseSetup` as normal.

NOTE: The cornering flow field is not laterally symmetric, so the case is always built full width. If the case is configured for half symmetry, `caseSetup` automatically promotes it to a full domain when `RUN_CORNERING` is `True`.

6.2 The Corner Centre

With `CORNER_CENTER` set to `default`, the corner centre is derived from the reference centre of rotation `REFCOR` (the wheelbase centre, at `y = 0`, projected to the ground). The centre is placed a lateral distance of `CORNER_RADIUS` to the side set by `CORNER_DIR`:

- `CORNER_DIR = left` - centre at `REFCOR` with `y = REFCOR_y - CORNER_RADIUS`
- `CORNER_DIR = right` - centre at `REFCOR` with `y = REFCOR_y + CORNER_RADIUS`

Deriving the centre from `REFCOR` makes the car pivot about its wheelbase centre rather than the front axle. The rotation axis is the tilted ground normal, the same normal used by the ride-height attitude, so the rotating frame stays aligned with the floor when the domain is pitched or rolled. Set `CORNER_CENTER` to three explicit floats to override the derived centre entirely.

6.3 Wheel Rotation in a Corner

In the rotating frame each wheel sits at a different distance from the corner axis, so the inner wheels travel a shorter path than the outer wheels. `caseSetup` sets each wheel's rotating-wall speed to the local ground speed at that wheel, equal to the frame angular velocity multiplied by the horizontal distance from the corner axis to the wheel centre. Inner wheels therefore spin slower and outer wheels faster than they would in a straight-line run at `INLET_MAG`.

6.4 Cornering Driven by the Ride-Height Map

Cornering can be combined with a ride-height study so that each ride-height point runs at the corner that physically matches its attitude. When both `RUN_RIDE_HEIGHT` and `RUN_CORNERING` are `True`, `caseSetup` looks for two optional columns in the ride-height file and applies them per point:

- `corner_radius` `[[float,m]]` - per-point corner radius, overrides `CORNER_RADIUS` for that child case, also accepted as `cornerradius`, `corner_r` or `radius` (float)
- `corner_dir` `[str]` - per-point turn direction, overrides `CORNER_DIR` for that child case, also accepted as `cornerdir`, `turn_dir` or `direction` (left, right)

An example ride-height file pairing each attitude with its corner is shown below.

```
point,fl,fr,rl,rr,yaw,steer,corner_radius,corner_dir
1,0,0,0,0,0,0,100,left
2,-10,-10,0,0,0,-5,100,left
3,-20,-20,0,0,0,-5,80,right
```

Each child case is generated with its own corner radius and direction, so it produces the matching corner centre, frame angular velocity and per-wheel rotation. If the `corner_radius` column is absent, every point falls back to the base `CORNER_RADIUS`, and likewise for `corner_dir`. A value that cannot be parsed, or a direction that is not `left` or `right`, is reported as a warning and falls back to the base setting for that point.

NOTE: A valid base `CORNER_RADIUS` is still required even when a `corner_radius` column is provided. The base value is validated before the child cases are generated and is used as the default for any point that does not supply its own radius.

6.5 Domain Validity

A rectangular box only approximates a curved tunnel when the turn radius is large relative to the box width. `caseSetup` enforces

```
CORNER_RADIUS >= CORNER_CLEARANCE_FACTOR * (DOMAIN_SIZE_y / 2)
```

and stops with an error reporting the minimum admissible radius if the check fails. The domain is never resized automatically. If a radius is rejected, increase `CORNER_RADIUS`, reduce the lateral domain size `DOMAIN_SIZE_y`, or lower `CORNER_CLEARANCE_FACTOR`. When cornering is driven by the ride-height map, this check is applied to every point so an unsuitable per-point radius is caught early.

6.6 Workflow Summary

- Set `RUN_CORNERING` to `True`, and provide `CORNER_RADIUS` and `CORNER_DIR`.
- Leave `CORNER_CENTER` as `default` to pivot about the wheelbase centre, or set three floats to place the centre explicitly.
- To sweep attitudes and corners together, also set `RUN_RIDE_HEIGHT` to `True` and add `corner_radius` and `corner_dir` columns to the ride-height file.

- Run `caseSetup` as normal. The case is built full width, the SRF rotation is written, the wheels are set to their local corner speeds, and the run uses `CORNER_SOLVER`.

6.7 Troubleshooting

- `CORNER_RADIUS` too small for this domain - the clearance check failed. Increase `CORNER_RADIUS`, reduce `DOMAIN_SIZE y`, or lower `CORNER_CLEARANCE_FACTOR` to at least the reported minimum radius.
- `CORNER_RADIUS` must be a positive number - `RUN_CORNERING` is `True` but no valid base radius was given. Set `CORNER_RADIUS` directly, or provide a `corner_radius` column in the ride-height file together with a base value.
- `CORNER_DIR` must be `left` or `right` - the turn direction was not recognised. Use `left` or `right`, or correct the `corner_dir` column in the ride-height file.
- `Per-point corner value ignored` - a `corner_radius` or `corner_dir` entry could not be parsed and the base setting was used for that point. Check the column values in the ride-height file.

7 Post-Processing Images (pvPost)

pvPost is the automated ParaView image generator. It reads the converged solution, builds the standard set of geometry, surface-contour, slice, line-integral-convolution (LIC) and iso-surface images, and writes them to `postProcessing/images`. It runs head-less through **pvbatch** and is driven entirely by the **pvPostSetup** file copied into each case, so no manual ParaView session is required.

7.1 Inputs and Outputs

pvPost expects the solution to have been exported to EnSight format. It reads

```
EnSight/<caseName>.case
```

and uses the latest available time step. Iso-surface images additionally read the `iso*.vtp` files written under `postProcessing/surfaces/<latestTime>`. All images are written as `.jpeg` into

```
postProcessing/images/<variable>_<imageType>/
```

one sub-folder per variable and image type. Slice images include the slice normal, position and an index in the file name so they sort in sweep order.

7.2 Running pvPost

pvPost is a ParaView script and must be launched with **pvbatch** (or **pvpython**), not the system **python**, so that the `paraview.simple` modules are available. From inside the case directory:

```
cd <caseName>
pvbatch --mpi /path/to/postUtilities/pvPost.py >> log.pvpost
```

On the cluster this is wired into the post-processing job through **clusterDict** and runs automatically after the solve. There is one optional flag:

- **-meshOnly** - generates only the mesh slice images and skips all field images. Use this to inspect the mesh before a solution exists.

```
pvbatch --mpi /path/to/postUtilities/pvPost.py --meshOnly >> log.pvpostMesh
```

7.3 The pvPostSetup File

caseSetup copies a **pvPostSetup** file into every case. Edit it to choose which image sets are produced and how the slices are taken. The default file is shown below.

```
[PV_POST_MAIN]
CASE_NAME = default
CAMERA_VIEWS = default
VARIABLE_DICT = default
WHEEL_BASE = default
BACKGROUND_COLOR = default
RESOLUTION = default
GEOM_IMG = True
SURFACE_IMG = True
SLICE_IMG = True
SLICE_LIC_IMG = True
ISO_SURFACE_IMG = True
CREATE_SCALAR_BAR = True

[GEOM]
VIEWS = default

[SURFACE]
VIEWS = default
VARIABLES = default

[SLICE]
VIEWS = default
NORMALS = default
NSLICES = default
```

```

SLICE_RANGE = default
VARIABLES = default
LIC_VARIABLES = default

[ISO_SURFACE]
VIEWS = default
VARIABLES = default
ISO_VALUES = default

[VARIABLES]
SURFACE_VARIABLES = default
SLICE_VARIABLES = default
ISO_VARIABLES = default

```

7.3.1 [PV_POST_MAIN]

- **CASE_NAME** [**str**] - case name used in image titles and file names, **default** uses the case directory name (**default**, or **str**)
- **CAMERA_VIEWS** [**str**] - path to a camera-views file, **default** uses the bundled view definitions (**default**, or a file path)
- **VARIABLE_DICT** [**str**] - path to the variable dictionary that defines the plotted variables, their equations, labels, ranges and colour maps, **default** uses `postUtilities/default/defaultVariables` (**default**, or a file path)
- **RESOLUTION** [**str**] - working render resolution, **default** uses the built-in 16:9 setting (**default**)
- **GEOM_IMG**, **SURFACE_IMG**, **SLICE_IMG**, **SLICE_LIC_IMG**, **ISO_SURFACE_IMG** [**bool**] - master switches for each image set, set to **False** to skip that set (**True**, **False**)
- **CREATE_SCALAR_BAR** [**bool**] - draw the colour-bar legend on field images (**True**, **False**)

7.3.2 [SLICE]

The slice block controls both the field slices and the LIC slices. Each of **VIEWS**, **NORMALS**, **NSLICES** and **SLICE_RANGE** is a list, and the lists must all be the same length: entry *i* defines one slice plane sweep.

- **VIEWS** [**str**] - camera view used to render each plane (**default** = `Front Left LeftForward LeftBack Top`)
- **NORMALS** [**str**] - slice plane normal for each sweep (**default** = `X Y Y Y Z`)
- **NSLICES** [**int**] - number of evenly spaced slices in each sweep, **default** adapts to the symmetry of the case
- **SLICE_RANGE** [[**float,m**] [**float,m**]] - start and end coordinate of each sweep as [**min,max**], **default** derives sensible ranges from the front-axle reference and the reference dimensions
- **VARIABLES** [**str**] - field-slice variables to plot, **default** plots the standard pressure, velocity and vorticity set
- **LIC_VARIABLES** [**str**] - variables coloured underneath the LIC streamlines, **default** uses the standard set

A variable requested here that is not defined in the variable dictionary, or not present in the solution, is reported and skipped rather than stopping the run.

7.3.3 [SURFACE] and [ISO_SURFACE]

- [\[SURFACE\] VIEWS / VARIABLES](#) - camera views and surface-contour variables painted on the car body (e.g. [CpMean](#), wall shear).
- [\[ISO_SURFACE\] VIEWS / VARIABLES / ISO_VALUES](#) - camera views for the iso-surface renders. The iso-surfaces themselves ([isoQ](#), [isoCtp](#), ...) are produced by the solver's surface function objects and read from [postProcessing/surfaces](#); [pvPost](#) renders whatever [iso*.vtp](#) files it finds.

7.4 Variables and Camera Views

The variable dictionary defines every plottable variable as an equation over the OpenFOAM fields, together with a display label, a value range and a ParaView colour preset. The tokens [UREF](#), [LREF](#) and [WREF](#) in the equations are substituted with the case reference values before evaluation, so coefficients such as [CpMean](#) and [CptMean](#) are normalised consistently with the rest of the case. To use a custom set, point [VARIABLE_DICT](#) (and, if needed, [CAMERA_VIEWS](#)) at your own file.

7.5 Half-Symmetry Cases

When the case is built on half symmetry ([SIM_SYM = half](#)), [pvPost](#) automatically reflects the body surface, the internal volume and the iso-surfaces about the symmetry plane so the rendered images show the full car.

7.6 Cornering (SRF) Cases

For an SRF cornering case the time-averaged velocity stored in the solution is the rotating-frame relative velocity [UrelMean](#), not the absolute [UMean](#). [pvPost](#) detects a cornering case automatically from [CORNERING_SETUP](#) -> [RUN_CORNERING](#) and, when it is [True](#):

- rewrites every variable equation that references [UMean](#) (including its components [UMean_X/Y/Z](#) and the [UMean^2](#) term in [CptMean](#)) to read [UrelMean](#) instead, and
- uses [UrelMean](#) as the vector field for the LIC streamlines.

Variable names, labels and ranges are unchanged, so the same [pvPostSetup](#) works for both straight-line and cornering runs and the image file names stay consistent. A line is printed to [log.pvpost](#) confirming the substitution:

```
Cornering (SRF) case detected: using UrelMean in place of UMean.
```

NOTE: This requires [UrelMean](#) to be present in the solution. The cornering setup averages both [U](#) and [Urel](#), so [UrelMean](#) is available; if it is missing, those variables are reported as unavailable and skipped.

7.7 Performance

For speed, [pvPost](#) loads only the fields actually referenced by the active variable equations (plus the LIC vector) from the EnSight reader, dropping unused fields such as [phi](#), [k](#), [omega](#), [nut](#) and the [Prime2Mean](#) tensors before the cell-to-point conversion and slicing. It also reuses a single slice filter and calculator per slice normal, updating only the slice position as it sweeps, rather than rebuilding the pipeline for every slice. These changes reduce runtime and memory with no change to the images produced. The console reports which fields were loaded and which were skipped:

```
Loading only required fields: ['pMean', 'UMean', ...]
Skipping unused fields: ['phi', 'k', 'omega', ...]
```

7.8 Workflow Summary

- Export the converged solution to EnSight (`EnSight/<caseName>.case`).
- Edit `pvPostSetup` to choose the image sets (`GEOM_IMG`, `SURFACE_IMG`, `SLICE_IMG`, `SLICE_LIC_IMG`, `ISO_SURFACE_IMG`) and, if needed, the slice `NORMALS`, `NSLICES` and `SLICE_RANGE`.
- Run `pvPost` with `pbatch` from the case directory (automatic on the cluster). Add `-meshOnly` to render only the mesh.
- Collect the images from `postProcessing/images`. Cornering cases are handled automatically; no setup change is required.

7.9 Troubleshooting

- `ERROR! Unable to find EnSight File!` - no `EnSight/<caseName>.case` was found. Export the solution to EnSight before running `pvPost`.
- `The inputs in pvPostSetup is not correct for [SLICE]` - the `VIEWS`, `NORMALS`, `NSLICES` and `SLICE_RANGE` lists are not all the same length. Make every `[SLICE]` list have one entry per sweep.
- `<variable> not available, skipping` - the variable is not defined in the variable dictionary or not present in the solution. Check the spelling in `pvPostSetup` and confirm the field was written by the solver.
- `Could not determine required fields from variables, loading all` - field detection found nothing to load and fell back to loading every field. The run still completes; check that the variable dictionary equations reference real field names.
- `No iso-surface files found, skipping` - no `iso*.vtp` files exist under `postProcessing/surfaces`. Enable the iso-surface function objects in the solver setup if iso images are required.